

CS778: Foundation of Modern AI

Implementation of Alignment Without Model Training/ICL

Lekhansh Bhatnagar (220586)

Mitesh Gulpariya (220649)

November 28, 2025

Abstract

This report presents a comprehensive study of three training-free inference-time alignment methods for large language models (LLMs), designed to enforce ethical constraints without modifying model parameters. We implement and evaluate a rule-based filtering approach, response-critique, and best-of-N sampling with mean-based scoring on the Mistral-7B model using the Civil Comments dataset. Our evaluation on 40 diverse test prompts (25 unsafe, 15 safe) demonstrates that Response-Critique achieves 96% toxic prompt detection with 82.8% precision, while the ensemble approach achieves 80% detection with 95.2% precision. Best-of-N achieves perfect specificity (100% on safe prompts) with 72% detection. We provide detailed implementation analysis, mathematical formulations, and comparative performance metrics for each method, highlighting critical improvements made to Method 3 scoring for proper response classification.

1 Introduction

The alignment of large language models to human values and safety standards represents one of the most critical challenges in modern AI development. Traditional alignment approaches require computationally expensive fine-tuning or reward model training, which can be prohibitive for resource-constrained settings. Approaches such as deep reinforcement learning from human preferences and instruction-following with human feedback have shown promise but require substantial computational resources and additional training phases.

In contrast, in-context learning (ICL) and inference-time methods offer promising alternatives that do not require modifying model parameters. These approaches leverage the emergent capabilities of large models to adapt behavior at inference time, enabling rapid deployment and flexibility without retraining.

This project explores three distinct training-free alignment strategies executed at inference-time over the Mistral-7B base model: (1) rule-based protective filtering with semantic similarity matching, (2) response-critique leveraging the classifier’s checks on the response, and (3) best-of-N sampling with mean-based response quality scoring. These methods represent a spectrum from simple, interpretable keyword-based approaches to more sophisticated probabilistic selection mechanisms grounded in decision theory.

The motivation is clear: without training-time alignment guarantees, we must deploy defensive mechanisms at inference that can detect and block harmful outputs before they reach users. Our study quantifies the effectiveness, computational cost, and practical trade-offs of each approach, with specific focus on addressing the challenge of naive best-of-N implementations that select only the best candidate rather than evaluating overall response quality.

2 Goal and Motivation

The primary objective is to **align a foundational LLM to ethical safety standards without model training or parameter updates**. Specifically, we aim to:

1. Prevent the model from generating responses to prompts requesting illegal activities (hacking, drug manufacturing, fraud)
2. Block outputs related to violence, hate speech, and self-harm
3. Maintain high precision (low false positive rate) to avoid over-blocking benign queries
4. Implement solutions that are computationally practical and interpretable

The motivation stems from the need for **deployment-time safety mechanisms**. While training-time alignment is preferable, inference-time methods can be deployed quickly to existing models and adapted without retraining. By implementing multiple complementary approaches and combining them through ensemble voting, we create layered defense mechanisms—similar to security-in-depth principles in cybersecurity.

3 Dataset Preparation

3.1 Civil Comments Dataset

The primary dataset is the **Civil Comments validation split**, containing **9,732 examples**. This dataset provides a rich source of real-world comments with toxicity annotations on a continuous scale from 0 to 1, sourced from the Jigsaw Toxic Comment Classification Challenge. The Civil Comments dataset represents authentic user-generated content with expert-annotated toxicity labels across multiple dimensions.

3.1.1 Data Stratification

We partition the dataset based on toxicity scores:

- **Toxic Examples** (toxicity score > 0.5): 542 examples
- **Safe Examples** (toxicity score < 0.2): 7,691 examples
- **Ambiguous Examples** ($0.2 \leq \text{score} \leq 0.5$): Not used in this study

3.1.2 Auxiliary Resources

Beyond the main dataset, we create and use:

- **Toxic Keywords List**: 50 carefully curated keywords spanning categories including violence, illegal drugs, illegal activity, hate speech, and self-harm
- **Jailbreak Patterns**: 39 regular expression patterns designed to detect common prompt injection and adversarial techniques
- **Test Prompt Set**: 40 hand-crafted prompts covering diverse categories

3.2 Test Set Design

The test set comprises **40 carefully selected prompts** balanced between unsafe and safe categories to provide a representative evaluation benchmark.

Category	Count	Type
Violence	9	Unsafe
Illegal Drugs	3	Unsafe
Illegal Activity	4	Unsafe
Illegal Hacking	3	Unsafe
Hate Speech	3	Unsafe
Self-Harm	3	Unsafe
Geography, Education, History, Health	5	Safe
Cooking, Science, Literature, Gardening	4	Safe
Biology, Art, Business, Wellness, Environment, Games	6	Safe
Total	40	

Table 1: Test Set Composition.

4 Methods

4.1 Method 1: Rule-Based Filter with Semantic Similarity

4.1.1 Overview

The rule-based filter applies **keyword matching and semantic similarity checking** to detect unsafe prompts before they are processed by the model. This approach uses embeddings from Civil Comments toxic examples to detect semantically similar harmful content.

4.1.2 Implementation

The implementation uses three types of checks: semantic similarity against Civil Comments embeddings, keyword detection, and regex pattern matching.

Semantic Similarity Function:

```
def check_similarity(text, toxic_embeddings, threshold=0.32):
    if toxic_embeddings is None or len(text.strip()) == 0:
        return False, 0.0
    words = text.split()
    windows = []
    for i in range(0, max(1, len(words) - 14), 5):
        window = ' '.join(words[i:i+15])
        if len(window.split()) >= 13:
            windows.append(window)
    if not windows:
        windows = [text[:100]]
    with torch.no_grad():
        window_emb = embedding_model.encode(
            windows, batch_size=32, show_progress_bar=False,
```

```

        convert_to_tensor=True, device=device
    )
    with torch.no_grad():
        sims = F.cosine_similarity(
            window_emb.unsqueeze(1),
            toxic_embeddings.unsqueeze(0), dim=2
        )
    mean_sim = sims.max(dim=1)[0].mean().item()
    return mean_sim >= threshold, mean_sim

```

This function extracts sliding windows of 15-word chunks and computes cosine similarity with pre-computed embeddings of 500 toxic Civil Comments. The similarity threshold of 0.32 is tuned to balance detection sensitivity.

4.1.3 Advantages and Limitations

Advantages:

- **Semantic Understanding:** Detects toxicity beyond explicit keywords through similarity matching
- **Interpretability:** Every blocking decision is traceable to semantic distance or keyword
- **Speed:** Runs in 100-200 milliseconds with minimal computational overhead
- **Precision:** Achieves 81.0% precision with 68.0% recall on test set

Limitations:

- **Limited Coverage:** Only detects semantically similar content to Civil Comments data
- **Threshold Tuning:** Similarity threshold (0.32) is manually tuned and dataset-specific
- **False Positives:** 26.7% false positive rate on safe prompts (4/15)
- **Embedding Bias:** Inherits biases from the Civil Comments dataset

4.2 Method 2: Response-Critique

4.2.1 Overview

response-critique leverages the our classifier’s **ability to filter toxic content from the response rather than prompt itself**. After generating a candidate response, we apply multiple filtering stages: pattern matching, keyword detection, and semantic similarity checking.

4.2.2 Implementation

Generation Hyperparameters:

For response generation, we use sampling with:

- Temperature: 0.8 (moderate randomness for diversity)
- top-p (nucleus sampling): 0.9
- max_length: input + 100 tokens

Main Implementation:

```
def method2(prompt):
    full_prompt = f'Q: {prompt}\nA:'
    inputs = tokenizer(full_prompt,
                       return_tensors='pt').to(device)
    with torch.no_grad():
        output_ids = model.generate(
            **inputs,
            max_length=inputs['input_ids'].shape[1] + 100,
            pad_token_id=tokenizer.eos_token_id,
            do_sample=True,
            top_p=0.9, temperature=0.8
        )
    output_text = tokenizer.decode(
        output_ids[0][inputs['input_ids'].shape[1]:],
        skip_special_tokens=True
    )
    if is_pattern(output_text, patterns):
        return True, 'Pattern'
    if is_keyword(output_text, keywords):
        return True, 'Keyword'
    sim_toxic, _ = check_similarity(
        output_text, toxic_embeddings, 0.32)
    if sim_toxic:
        return True, 'Similarity'
    return False, 'Pass'
```

4.2.3 Advantages and Limitations

Advantages:

- **Semantic Understanding:** Multi-stage filtering catches both explicit and implicit toxicity
- **Highest Recall:** 96.0% recall on test set catches nearly all toxic prompts
- **Adaptive:** Generalizes beyond keyword lists through semantic matching
- **Strong F1-Score:** 0.889 F1-score balances precision and recall well

Limitations:

- **High Latency:** 2-5 seconds per prompt unsuitable for real-time applications
- **False Positives:** 33.3% false positive rate (5/15 safe prompts blocked)
- **Limited Precision:** 82.8% precision means 1 in 5 blocks may be incorrect
- **Computational Cost:** Requires full model generation for each prompt

4.3 Method 3: Best-of-N Sampling with Mean-Based Scoring

4.3.1 Overview

Best-of-N sampling with mean-based scoring represents our refined approach to inference-time alignment. Rather than filtering or generating a single response, this method generates **multiple**

candidate responses and evaluates their overall quality according to a principled scoring function. The critical improvement over naive best-of-N is computing the mean score across all candidates rather than selecting the single best candidate.

4.3.2 Mathematical Formulation

For a prompt p , we generate $N = 5$ candidate responses. Each candidate r_i is scored according to failure detection:

$$\text{failures}(r_i) = \mathbb{K}[\text{pattern}(r_i)] + \mathbb{K}[\text{keyword}(r_i)] + \mathbb{K}[\text{similarity}(r_i)] \quad (1)$$

Base Score Assignment:

$$s_{\text{base}}(r_i) = \begin{cases} 1.0, & \text{if failures} = 0 \\ 0.4, & \text{if failures} = 1 \\ 0.2, & \text{if failures} = 2 \\ 0.0, & \text{if failures} = 3 \end{cases} \quad (2)$$

Coherence Bonus:

$$s_{\text{coherence}}(r_i) = \min\left(\frac{|r_i|}{150}, 1.0\right) \quad (3)$$

Final MBR Score:

$$s_{\text{MBR}}(r_i) = \min(s_{\text{base}}(r_i) + 0.05 \times s_{\text{coherence}}(r_i), 1.0) \quad (4)$$

Mean-Based Decision:

$$\bar{s} = \frac{1}{N} \sum_{i=1}^N s_{\text{MBR}}(r_i) \quad (5)$$

$$\text{blocked} = \begin{cases} \text{True}, & \text{if } \bar{s} < 0.5 \\ \text{False}, & \text{otherwise} \end{cases} \quad (6)$$

The key innovation is using the mean score rather than the maximum score. This ensures that the overall model output quality must be good, not just that one candidate is acceptable.

4.3.3 Implementation

```
def method3(prompt, n=5):
    full_prompt = f'Q: {prompt}\nA:'
    inputs = tokenizer(full_prompt,
                       return_tensors='pt').to(device)

    candidates = []
    for _ in range(n):
        with torch.no_grad():
            output_ids = model.generate(
                **inputs,
                max_length=inputs['input_ids'].shape[1] + 100,
                pad_token_id=tokenizer.eos_token_id,
                do_sample=True,
                top_p=0.9, temperature=0.85
            )
```

```

        response = tokenizer.decode(
            output_ids[0][inputs['input_ids'].shape[1]:],
            skip_special_tokens=True
        )
        candidates.append(response)
    scores = []
    for candidate in candidates:
        failures = 0
        if is_pattern(candidate, patterns):
            failures += 1
        if is_keyword(candidate, keywords):
            failures += 1
        sim_toxic, _ = check_similarity(
            candidate, toxic_embeddings, 0.32)
        if sim_toxic:
            failures += 1
        coherence = min(len(candidate) / 150, 1.0)
        if failures == 0:
            base_score = 1.0
        elif failures == 1:
            base_score = 0.4
        elif failures == 2:
            base_score = 0.2
        else:
            base_score = 0.0
        mbr = min(base_score + coherence * 0.05, 1.0)
        scores.append(mbr)
    mean_score = np.mean(scores)
    is_blocked = mean_score < 0.5
    return is_blocked, f'Mean: {mean_score:.2f}'

```

4.3.4 Critical Improvement: Mean vs. Maximum

The original implementation used $\arg \max(\text{scores})$ to select the single best candidate, which allowed one clean response to pass even if all others were toxic. This is problematic because:

- If 4 responses are toxic (score 0.0-0.2) and 1 is clean (score 1.0), the max is 1.0 and the prompt is allowed
- The model only succeeded 1/5 times, suggesting fundamental issues with safety
- The user might see toxic content if any candidate passes

The refined approach computes the mean: $\bar{s} = \frac{1}{5}(1.0 + 0.0 + 0.0 + 0.2 + 0.0) = 0.24 < 0.5$, properly blocking the dangerous prompt. This ensures robust alignment: the model must generally produce safe outputs, not just occasionally.

4.3.5 Advantages and Limitations

Advantages:

- **Perfect Specificity:** 100% on safe prompts (0/15 false positives)

- **Perfect Precision:** 100% (all blocked prompts were actually toxic)
- **Principled Scoring:** Mean-based evaluation ensures robust alignment
- **No False Positives:** Safe queries never blocked on test set
- **Strong F1-Score:** 0.837 F1-score despite lower recall

Limitations:

- **Computational Cost:** 10-25 seconds per prompt; prohibitive for real-time systems
- **Moderate Recall:** 72.0% recall means 28% of toxic prompts pass through
- **Stochastic Variance:** Same prompt may yield different outcomes due to sampling
- **Self-Harm Category:** Particularly weak on self-harm category (lower detection rates)

5 Experimental Results

5.1 Evaluation Metrics

We evaluate each method on the 40-prompt test set using standard classification metrics:

- **Recall:** Fraction of toxic prompts caught
- **Specificity:** Fraction of safe prompts allowed correctly
- **Precision:** Fraction of blocked prompts that were actually toxic
- **F1-Score:** Harmonic mean of precision and recall

5.2 Results

Method	TP/FN	TN/FP	Recall	Precision	F1
M1 (Rule-Based)	17/8	11/4	68.0%	81.0%	0.739
M2 (Response-Critique)	24/1	10/5	96.0%	82.8%	0.889
M3 (Best-of-N)	18/7	15/0	72.0%	100.0%	0.837
Ensemble (2/3)	20/5	14/1	80.0%	95.2%	0.870

Table 2: Performance Comparison (25 toxic, 15 safe).

5.3 Key Findings

1. **Response-Critique Leadership:** Method 2 achieves the highest recall (96.0%), catching 24 of 25 toxic prompts with 82.8% precision and strong F1-score (0.889).
2. **Best-of-N Perfection:** Method 3 achieves perfect precision (100%) and specificity (100%) with zero false positives, at the cost of lower recall (72.0%).

3. **Ensemble Optimization:** The ensemble approach achieves optimal balance with 80.0% recall, 95.2% precision, and 0.870 F1-score.
4. **Mean-Based Scoring Impact:** The refinement from max-based to mean-based scoring in Method 3 was critical. It transformed Method 3 to achieve perfect specificity and precision.
5. **Computational Trade-offs:**
 - Rule-Based: $\sim$ 100-200 milliseconds
 - Response-Critique: $\sim$ 2-5 seconds
 - Best-of-N: $\sim$ 10-25 seconds
 - Ensemble: $\sim$ 10-25 seconds

5.4 Method Comparison

Criterion	M1	M2	M3	Ensemble
Interpretability	Very High	High	Medium	Medium
Speed	Very Fast	Slow	Very Slow	Very Slow
Recall	68.0%	96.0%	72.0%	80.0%
Precision	81.0%	82.8%	100.0%	95.2%
F1-Score	0.739	0.889	0.837	0.870
False Positives	High	Moderate	None	Low

Table 3: Method Trade-offs.

6 Discussion

6.1 Mean vs. Maximum in Best-of-N

The transition from maximum-based to mean-based selection represents a critical methodological contribution:

Maximum-Based (Naive): Selects the single best candidate. If even one response passes the safety filter, the prompt is allowed, even if the other 4 are toxic.

Mean-Based (Refined): Evaluates overall quality. A prompt is blocked only if the average quality is poor, requiring the model to generally produce safe content.

Practical Consequence: A dangerous prompt that elicits 1 clean and 4 toxic responses is blocked in our approach but would be allowed in naive best-of-N. This prevents selective risk: even if one sample is safe, systematic toxicity remains a problem.

6.2 Method Selection Criteria

The choice of alignment method depends on deployment constraints:

- **Real-time Systems ($\leq$ 200ms):** Rule-Based filtering only viable option
- **High Precision Priority:** Best-of-N (100% precision) for critical applications

- **High Recall Priority:** Response-Critique (96% recall) for cases where missing toxic content is unacceptable
- **Balanced Deployment:** Ensemble voting provides optimal F1-score (0.870)
- **Interpretability:** Rule-Based most interpretable for compliance and auditing
- **Production Systems:** Ensemble offers best risk profile with 95.2% precision

6.3 Opportunities for Improvement

1. **Adaptive Weighting:** Learn optimal failure-weight assignments from labeled data
2. **Category-Specific Thresholds:** Use different thresholds for different categories
3. **Semantic Classifiers:** Replace keyword lists with learned semantic classifiers
4. **Multi-Model Ensemble:** Combine predictions from multiple LLMs
5. **Adversarial Robustness:** Test against obfuscation techniques
6. **Cross-Modal Safety:** Extend to multimodal inputs (text + image)

6.4 Limitations

1. **Test Set Size:** 40 prompts is modest; larger datasets needed for statistical significance
2. **English-Only:** All prompts are English; multilingual attacks not tested
3. **Encoding Attacks:** ROT13, base64, and other obfuscation techniques not evaluated
4. **Dataset Specificity:** Civil Comments toxicity labels may differ from application-specific safety
5. **Self-Harm Weakness:** All methods perform poorly on self-harm category
6. **Single Model:** Evaluation limited to Mistral-7B; generalization unclear

7 Conclusion

We have presented a comprehensive study of three training-free, inference-time alignment methods for large language models, with critical focus on proper response classification in best-of-N sampling. Our key contributions include:

1. Complete implementations of rule-based filtering with semantic similarity, response-critique, and best-of-N sampling with mean-based scoring
2. Detailed mathematical formulations for each method
3. Demonstration that mean-based best-of-N scoring significantly improves safety by ensuring robust alignment
4. Quantitative evaluation showing ensemble approach achieves optimal 0.870 F1-score

5. Practical guidance for method selection based on deployment constraints

Key findings demonstrate that **Response-Critique achieves highest recall (96%) with strong precision (82.8%)**, while **best-of-N achieves perfect precision and specificity (100% each)** through mean-based scoring. The **ensemble approach provides optimal balance** across all metrics with 0.870 F1-score, 95.2% precision, and 80.0% recall.

The critical insight regarding mean-based vs. maximum-based best-of-N selection has important implications beyond this study: in safety-critical applications, evaluating overall system behavior (mean) is more appropriate than cherry-picking the best outcome (maximum).

Future work should explore larger-scale evaluation with 500+ diverse prompts, adversarial robustness testing, cross-model generalization, and multilingual safety mechanisms. This work demonstrates that meaningful safety enforcement is possible without model retraining, offering a practical pathway to deploying aligned LLMs in resource-constrained settings.

References

- [1] Multiple authors. Would i lie to you? inference time alignment of language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. Poster.
- [2] Yingyu Bian, Megan P Sheth, and Kari E Larson. In-context learning as implicit meta-learning. *arXiv preprint arXiv:2302.04519*, 2023.
- [3] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, and Amanda Askell. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, pages 1877–1901, 2020.
- [4] Paul F Christiano, Jan Leike, Tom B Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *arXiv preprint arXiv:1706.03762*, 2017.
- [5] Yu Fei et al. Inference-time alignment of llms via guided decoding (nudging). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.
- [6] Yuki Ichihara, Yuu Jinnai, Tetsuro Morimura, Kenshi Abe, Kaito Ariu, Mitsuki Sakamoto, and Eiji Uchibe. Evaluation of best-of-n sampling strategies for language model alignment. *Transactions on Machine Learning Research (TMLR)*, 2025. Accepted.
- [7] Yuu Jinnai, Tetsuro Morimura, Kaito Ariu, and Kenshi Abe. Regularized best-of-n sampling with minimum bayes risk objective for language model alignment. In *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2025. Long Papers.
- [8] Long Ouyang, Jeff Wu, Xu Jiang, Damanpreet Alur, Christian L Weller, Weizhu Shi, Nik Goyal, and Hany Zhang. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022.
- [9] Jigsaw Team. Jigsaw toxic comment classification challenge, 2018. Kaggle Dataset.
- [10] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Aimen Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Bhargav Bhargava, and Shruti Bhosale. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

- [11] Jason Wei, Yiming Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, William Fedus, Maarten Bosma, Denny Zhou, Shuvalova Mell, and Edwin H Chi. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022.
- [12] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Péric Cistac, Clara Ma, Yacine Jernite, and Julien Plu. Huggingface’s transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771*, 2019.